

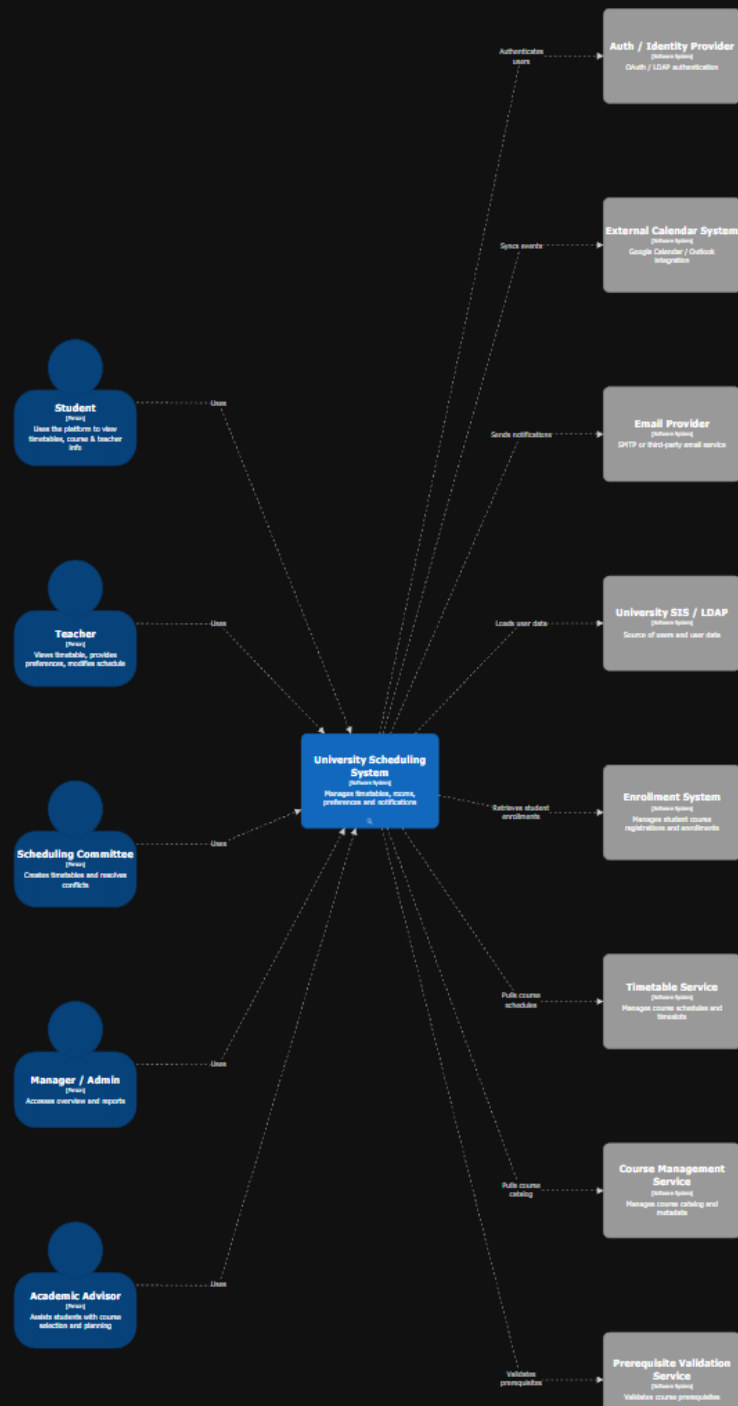
SCHEDULE 1

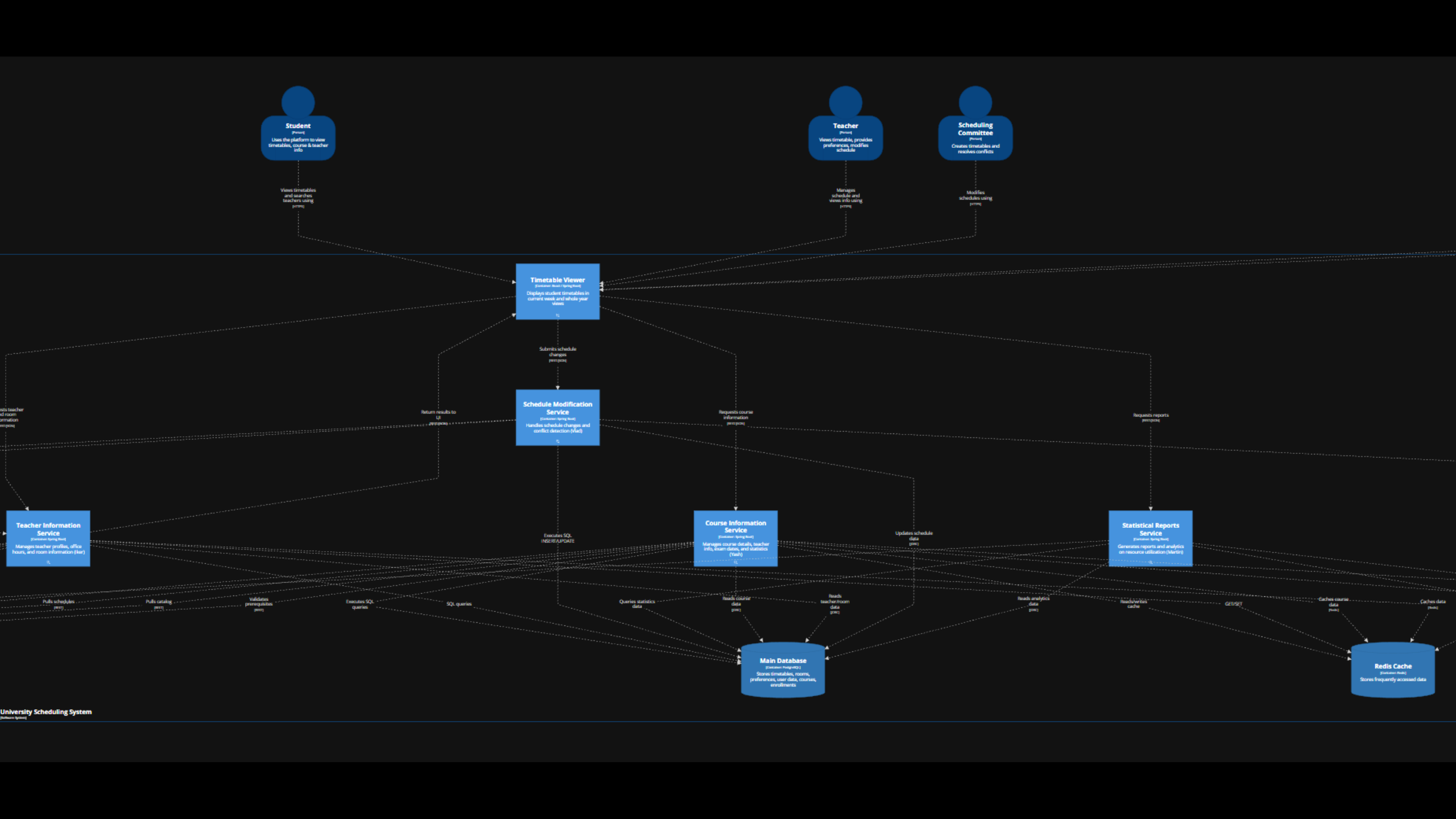
Team: EXA2

Ayat Idayatov, Emil Farzaliyev, Ivan Tregub, Adriana González Nieves

University wants to changes the login system

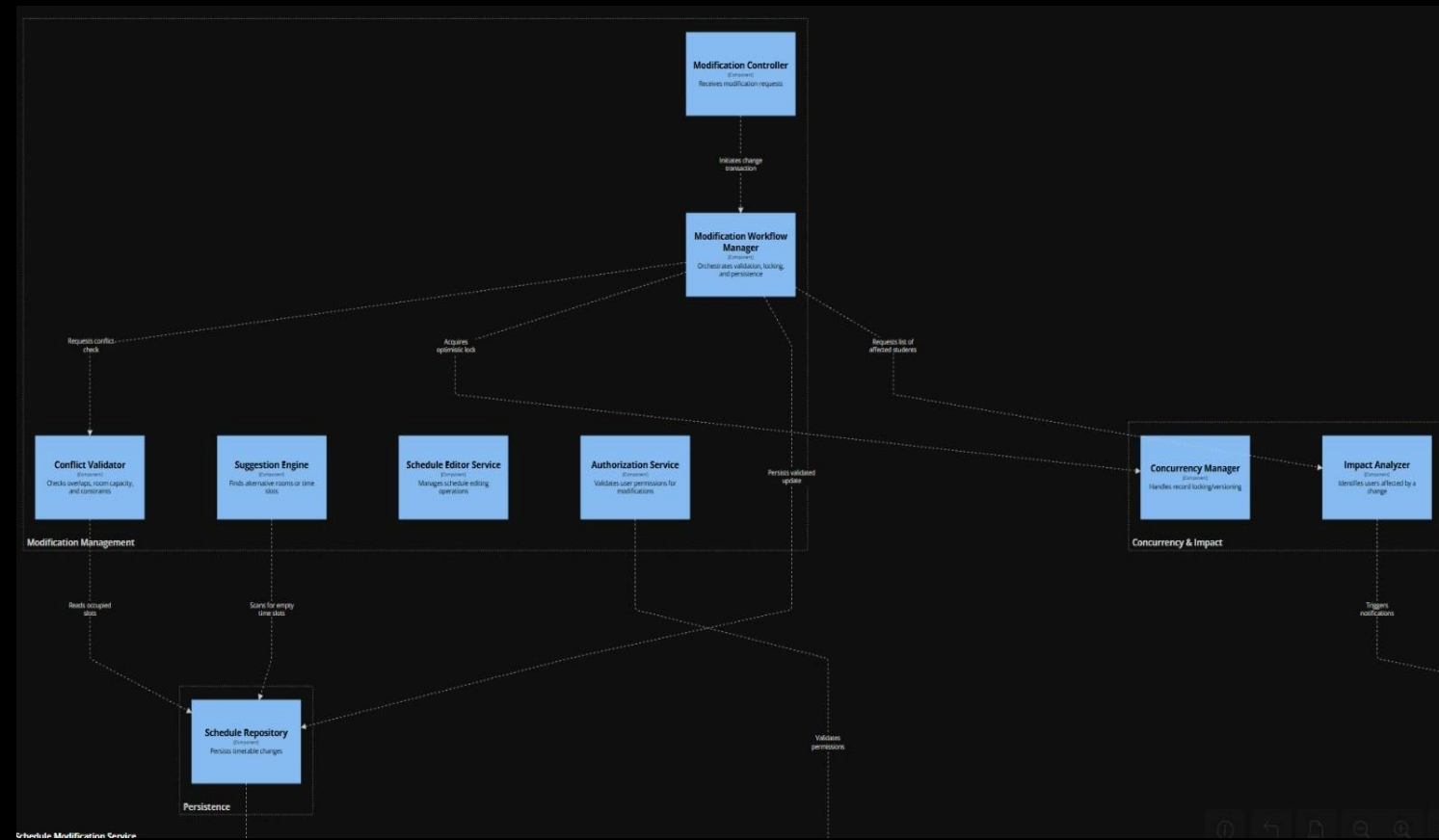
- Source of Stimulus: Developer
- Stimulus: University changes the Auth/Identity Provider
- Artifact: System-level auth integration and containers/components that enforce permissions, e.g.:
scheduleModificationContainer.authorizationService, role validation inside teacherInfoContainer (Auth/Role Validator), and the system's connection to authProvider.
- Environment: Design-time.
- Response: Developers update auth integration (configuration/adapter + role mapping), update affected services, then test and deploy the change to staging and production.
- Measure: Change implemented + deployed in ≤ 3 developer-days





Concurrency conflict during Timetable Modification

- Source of Stimulus: Scheduling Committee member
- Stimulus: Two committee members try to modify the same course timeslot/room at nearly the same time (concurrent update).
- Artifact: Schedule Modification Service - specifically Modification Workflow Manager + Concurrency Manager + Schedule Repository
- Environment: Production, during timetable-finalization period.
- Response: The system saves the first change. When the second person tries to save, the system notices the schedule was already changed by someone else, so it does not overwrite it. Instead, it shows a “someone updated this” message and loads the newest schedule so the second person can try again. The event is logged.
- Measure: 100% of conflicting concurrent edits are detected and rejected. Conflict response returned within ≤ 2 seconds



Modifiability – Ease of Feature Extension (Schedule Export)

- Source of Stimulus: University administration / product owner
- Stimulus: Add a feature to export student schedules to Google Calendar / iCal.
- Artifact:
 - timetableViewer (Timetable Viewer)
 - loadBalancer (Nginx / AWS ALB)
 - calendarIntegrationService (new)
 - scheduleModificationContainer (Schedule Modification Service)
 - externalCalendar (External Calendar System)
- Environment: Production, normal operation.
- Response: The Timetable Viewer sends an export request via the Load Balancer. The Calendar Integration Service fetches schedule data from the Schedule Modification Service (REST/JSON) and creates/updates events in the External Calendar System.
- Measure: Feature added by introducing a new service; existing services remain unchanged (no regression).

Modifications to the C4 Model

- Container Level:

A new container `CalendarIntegrationService` was added to the model. This change is reflected in the Container View.

- Relationships:

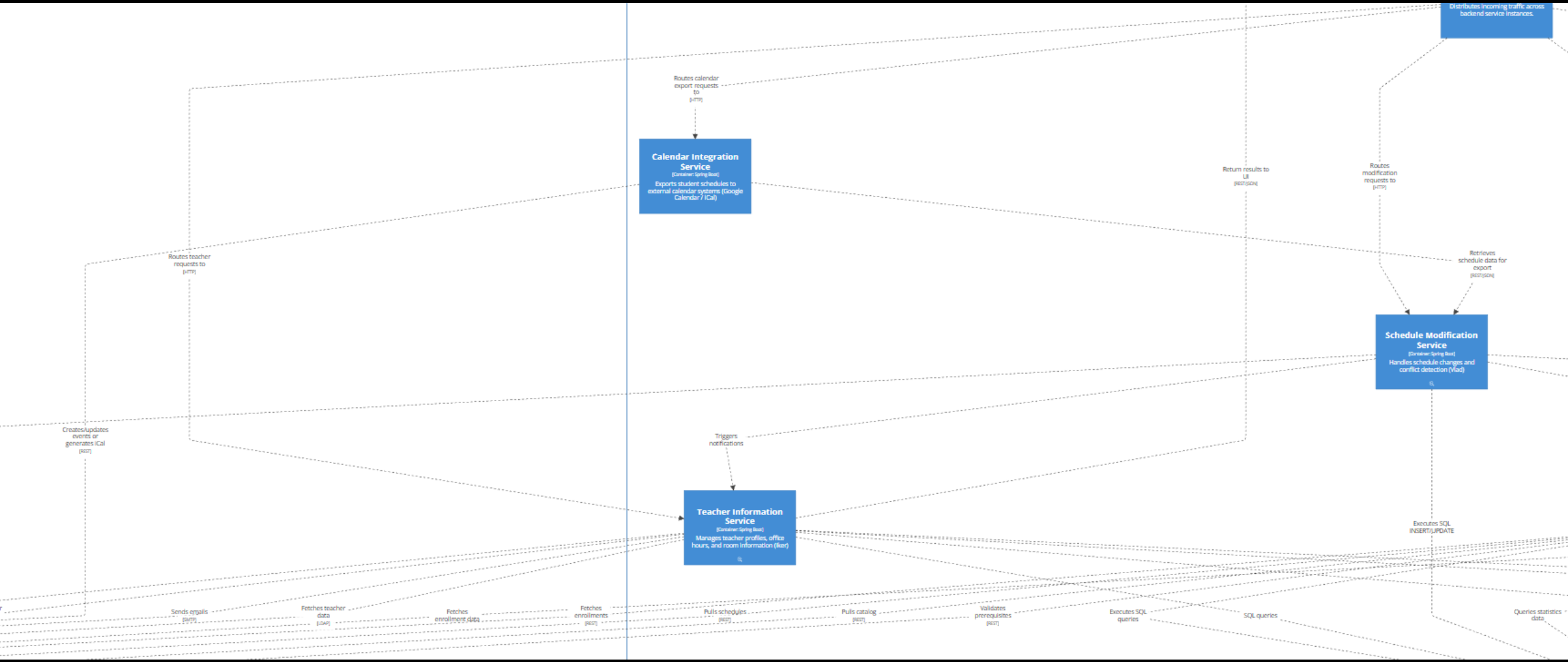
New relationships were introduced between the Load Balancer and the Calendar Integration Service, between the Calendar Integration Service and the Schedule Modification Service, and between the Calendar Integration Service and the External Calendar System. These changes are visible in both the System Context View and the Container View.

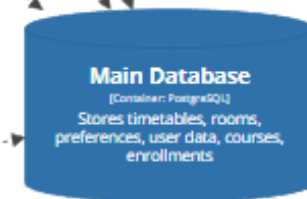
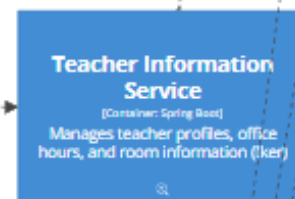
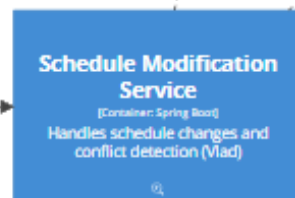
- Deployment:

The new service was added as a container instance in the Development environment and deployed in two availability zones in the Production environment. These changes are reflected in the Development and Production Deployment Views.

- Dynamic View:

A new dynamic view was added to document the runtime interaction when a student exports a schedule to an external calendar.





Retrieves
schedule data for
export
(REST/JSON)

Updates schedule
data
(JSON)

Executes SQL
INSERT/UPDATE

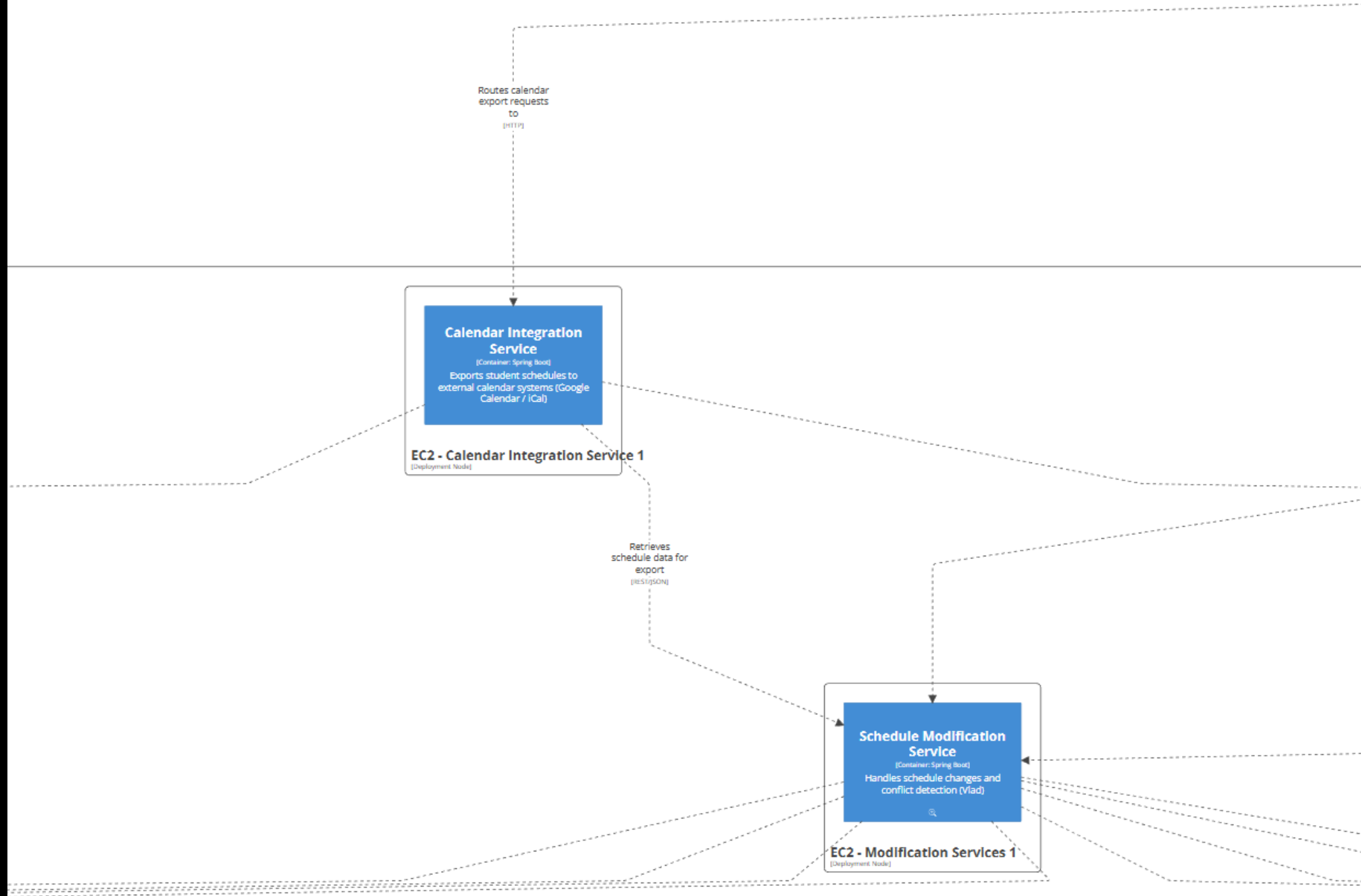
Triggers
notifications

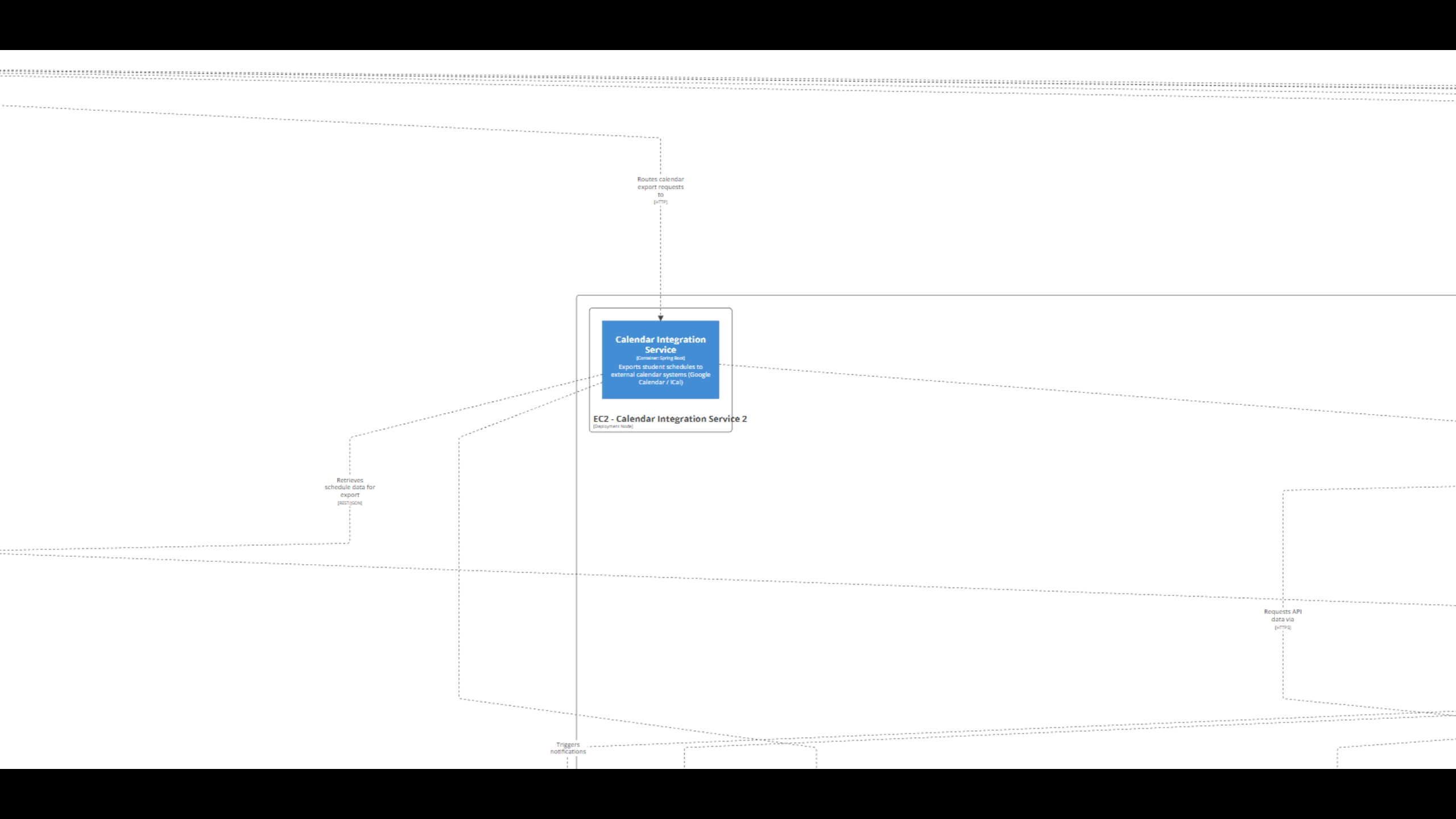
Reads
teacher/room
data
(JSON)

SQL queries

Reads course
data
(JSON)

Executes SQL
queries





Routes calendar
export requests
to
(HTTP)



EC2 - Calendar Integration Service 2
(Deployment Node)

Retrieves
schedule data for
export
(POST/JSON)

Requests API
data via
(GET/JSON)

Triggers
notifications



First day of semester, traffic spikes to 5,000 concurrent students accessing "Timetable Viewer"

- **Source of Stimulus:** Student
- **Stimulus:** 5,000 concurrent users attempt to view their timetables simultaneously at 9:00 AM on the first day of the semester.
- **Artifact:** `universitySchedulingSystem.timetableViewer`, `universitySchedulingSystem.loadBalancer (New)`, and backend services (`courseInfoContainer`, `teacherInfoContainer`).
- **Environment:** Production, peak load, high concurrency.
- **Response:** The `loadBalancer` intercepts the massive influx of requests from the `timetableViewer` (UI) and distributes them across multiple backend service instances using a round-robin strategy. This prevents any single service instance from becoming a bottleneck and crashing.
- **Measure:** Average page load time remains under 2 seconds with 0% error rate (no timeouts).

Updated diagram

